

MAESTRIA GESTIÓN Y ANALITICA DE DATOS

GESTIÓN DE DATOS

Objetivo de aprendizaje. Diseñar e implementar una arquitectura de datos gobernada y de alta calidad, desde el almacenamiento hasta el respaldo, que soporte de forma segura, trazable y auditable el sistema de IA de scoring crediticio y detección de fraude de NeoCredit S.A., aplicando los principios de gobernanza de datos que exige el panorama regulatorio mundial de la IA.

Recursos: MySQL Workbench, Firebase, Lucidchart, Google Colab, Github, Docker Desktop, DataHub, Duplicati y el dataset “solicitudes_credito_neocredit.csv”

Profesora: Mg. Adriana Collaguazo

Grupo 3

Integrantes:

Anastasia Minina

Denis Chancay

Cristhian Burbano

Erick Figueroa

Proyecto: Gobernanza y calidad de datos

Escenario: NeoCredit S.A.

NeoCredit S.A. es una fintech latinoamericana que ofrece cuentas digitales, tarjetas de crédito y microcréditos en línea. Para agilizar sus aprobaciones, la empresa está desarrollando un motor de scoring crediticio y detección de fraude basado en IA que decide, de forma automática, a qué clientes otorgar crédito y qué transacciones bloquear. La directiva de NeoCredit conoce bien el caso Equifax revisado en el Taller 4: una filtración que expuso datos de 147 millones de personas por fallas de cifrado, permisos excesivos y falta de monitoreo. NeoCredit no quiere repetir ese error, y además necesita que su modelo de IA sea auditable ante reguladores y clientes. Por eso contrató al equipo de estudiantes de la Maestría para diseñar la arquitectura de datos completa que soportará este sistema de IA de forma segura y confiable.

1. Arquitecturas de almacenamiento

Diseñar y justificar una arquitectura híbrida: un modelo relacional (MySQL) para datos transaccionales del núcleo del negocio (clientes, cuentas, solicitudes de crédito, transacciones) y un modelo NoSQL (Firestore) para datos no estructurados que alimentan al modelo de IA (registros de comportamiento, texto de soporte al cliente, resultados de scoring).

Para satisfacer las necesidades operativas, analíticas y de seguridad de NeoCredit, se propone implementar una arquitectura de persistencia políglota (híbrida). Este enfoque separa estratégicamente el núcleo transaccional del entorno de Inteligencia Artificial, asegurando rendimiento, escalabilidad y mitigación de riesgos. La justificación de los modelos seleccionados es la siguiente:

1. Modelo Relacional (MySQL) para el núcleo transaccional: Se destinará MySQL para la gestión de datos críticos estructurados, tales como clientes, cuentas, solicitudes de crédito y transacciones financieras.

Integridad y Consistencia (ACID): En el sector Fintech, es innegociable que las transacciones y los saldos sean exactos. MySQL garantiza el cumplimiento estricto de las propiedades ACID, asegurando que no existan operaciones incompletas o datos huérfanos gracias a su estructura normalizada y restricciones de llaves foráneas.

Cumplimiento Normativo: Facilita la generación de reportes regulatorios estándar requeridos por las entidades financieras, manteniendo un control estricto sobre la información demográfica y financiera estática.

- 1. Separación de Perfil Financiero (3FN):** En lugar de mantener todos los atributos en una única tabla de clientes, se ha separado la entidad **Clientes** (datos demográficos estáticos como nombre, edad, ciudad) de la entidad **Perfil_Financiero** (score externo, deuda actual). Esto cumple con la 3FN, ya que los datos financieros dependen de la actividad crediticia y cambian con mayor frecuencia, evitando anomalías de actualización.
- 2. Integridad Referencial:** Todas las tablas secundarias (**Cuentas**, **Transacciones**, **Solicitudes_Credito**) utilizan llaves foráneas (**FOREIGN KEY**) vinculadas al

`id_cliente` o `id_cuenta`. Esto asegura, por ejemplo, que no pueda existir una transacción huérfana de una cuenta que no existe.

3. **Resultados Básicos:** Se mantiene una tabla `Resultados_Modelo_IA` en este entorno transaccional *únicamente* para guardar el estado final (Aprobado/Rechazado) y el flag de fraude, permitiendo que el sistema operativo central (core bancario) funcione fluidamente. El detalle y los logs de la IA se delegan al entorno NoSQL.

2. Modelo NoSQL (Firestore) para el ecosistema de IA: Se implementará Firestore (base de datos documental) para gestionar datos semiestructurados y no estructurados, como registros de comportamiento (logs de navegación), textos de soporte al cliente y los resultados detallados del scoring.

Flexibilidad de Esquema: Los modelos de Machine Learning evolucionan rápidamente. Firestore permite ingerir nuevos tipos de datos (por ejemplo, nuevas variables de comportamiento en la app) sin necesidad de rediseñar esquemas rígidos ni causar tiempos de inactividad (downtime).

Escalabilidad y Rendimiento: La captura de interacciones de usuarios genera grandes volúmenes de datos a alta velocidad. Firestore escala horizontalmente de forma nativa para soportar esta carga analítica sin degradar el rendimiento de la base de datos transaccional principal (MySQL).



3. Seguridad, Auditoría y Prevención (Caso Equifax): El diseño de esta arquitectura híbrida responde directamente a las lecciones del caso Equifax:

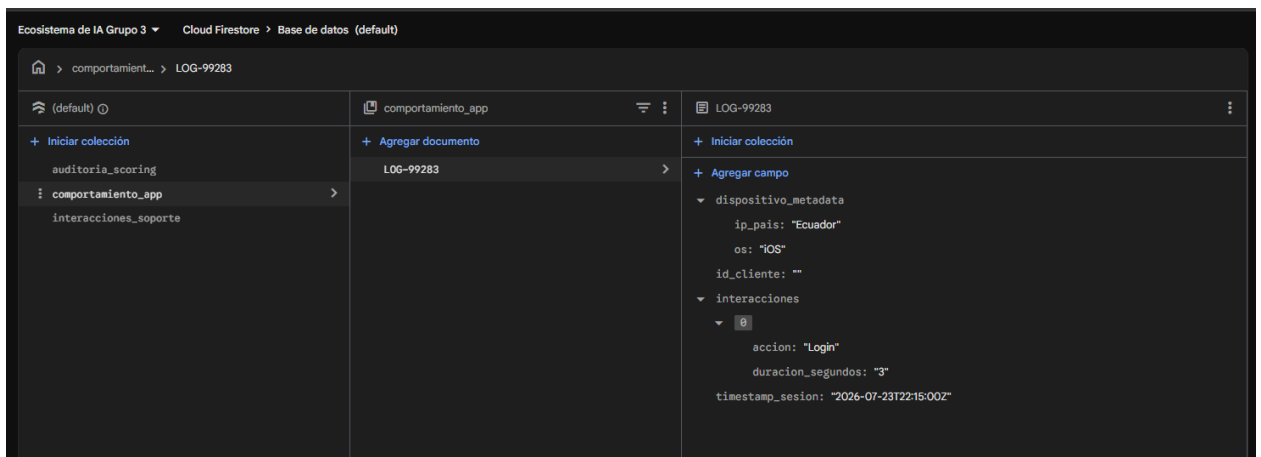
Aislamiento de Entornos: Al separar físicamente la base de datos transaccional de la analítica, se reduce drásticamente la superficie de ataque. Una vulnerabilidad en los permisos del entorno de IA o de lectura masiva de logs no comprometerá la base de datos central que contiene los saldos y transacciones.

Trazabilidad Exigida: Almacenar los "resultados de scoring" como documentos JSON inmutables en Firestore permite guardar el estado exacto de las variables, la versión del algoritmo y los pesos que la IA consideró en el momento de tomar una decisión. Esto garantiza la **auditabilidad total** ante los reguladores y clientes, permitiendo explicar de manera transparente por qué se aprobó o rechazó un crédito específico.

Se han definido tres colecciones principales:

Colección 1: `comportamiento_app` (Registros de comportamiento) Almacena los logs de navegación del cliente. Su esquema flexible permite agregar nuevas métricas (como tiempo en pantalla o clics) sin alterar la estructura.

```
{
  "_id": "LOG-99283",
  "id_cliente": "CLI-3535",
  "timestamp_sesion": "2026-07-23T22:15:00Z",
  "dispositivo_metadata": {
    "os": "iOS",
    "ip_pais": "Ecuador"
  },
  "interacciones": [
    {"accion": "login", "duracion_segundos": 3},
    {"accion": "revisar_tasas_credito", "duracion_segundos": 120}
  ]
}
```



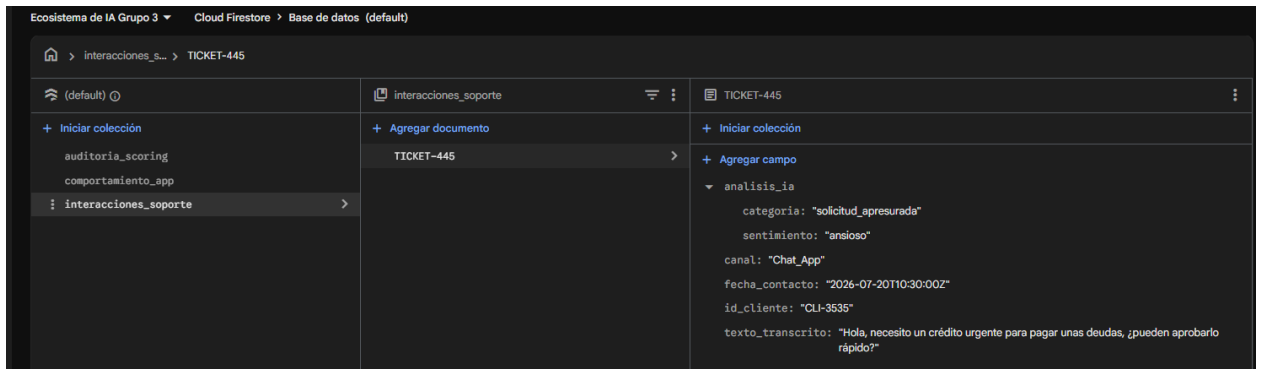
Colección 2: `interacciones_soporte` (Texto de soporte al cliente) Captura texto libre de chats o correos. Es fundamental para modelos de Procesamiento de Lenguaje Natural (NLP) que detecten urgencias o intentos de ingeniería social.

```
{
  "_id": "TICKET-445",
  "id_cliente": "CLI-3535",
  "fecha_contacto": "2026-07-20T10:30:00Z",
  "canal": "Chat_App",
  "texto_transcrito": "Hola, necesito un crédito urgente para pagar unas deudas, ¿pueden aprobarlo rápido?",
}
```

```

"analisis_ia": {
  "sentimiento": "ansioso",
  "categoria": "solicitud_apresurada"
}
}

```

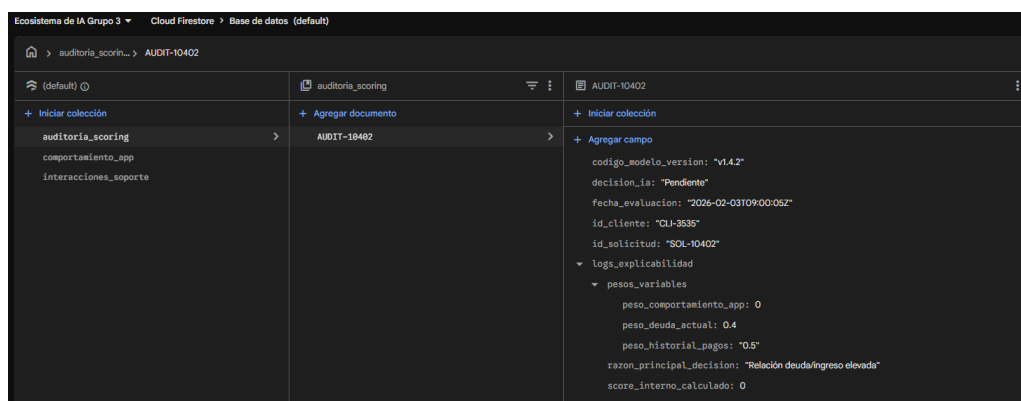


Colección 3: auditoria_scoring (Resultados de scoring y explicabilidad) Esta colección es la clave para la **auditoría exigida por los reguladores**. Almacena una "fotografía" inmutable de cada decisión de la IA, incluyendo la versión del algoritmo y la justificación exacta, algo imposible de manejar eficientemente en una tabla SQL tradicional.

```

{
  "_id": "AUDIT-10402",
  "id_solicitud": "SOL-10402",
  "id_cliente": "CLI-3535",
  "fecha_evaluacion": "2026-02-03T09:00:05Z",
  "decision_ia": "Pendiente",
  "codigo_modelo_version": "v1.4.2",
  "logs_explicabilidad": {
    "score_interno_calculado": 620,
    "probabilidad_fraude_porcentaje": 2.5,
    "razon_principal_decision": "Relación deuda/ingreso elevada según Perfil Financiero",
    "pesos_variables": {
      "peso_historial_pagos": 0.5,
      "peso_deuda_actual": 0.4,
      "peso_comportamiento_app": 0.1
    }
  }
}
}

```

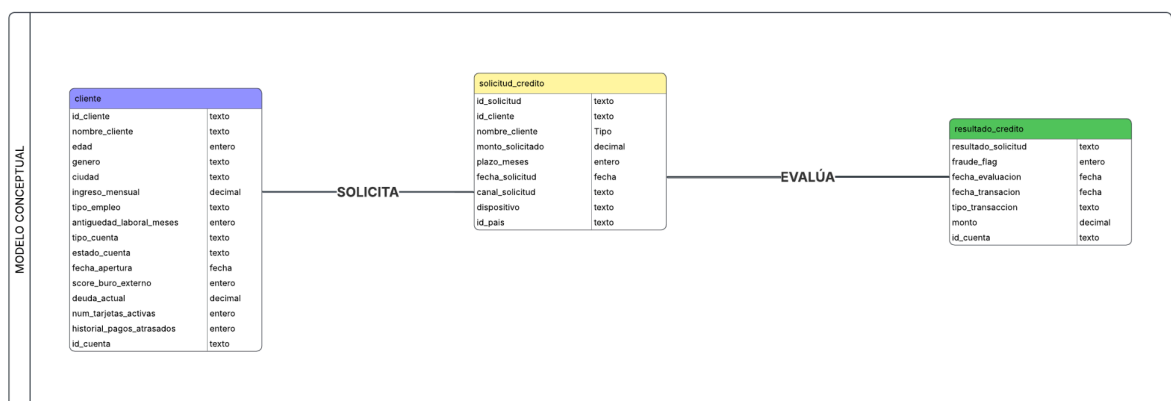


2. Modelado y normalización

Modelar las entidades Clientes, Cuentas, Transacciones, Solicitudes de Crédito y Resultados del Modelo de IA, aplicando normalización hasta 3FN en el modelo relacional.

Modelo conceptual

Iniciamos creando nuestro modelo conceptual de la base de datos, para esto hemos definido 3 tablas que contienen los atributos separados a nuestra consideración en base al dataset de ejemplo, las tablas mencionadas son las siguientes: cliente, solicitud_credito y resultado_solicitud. En esta primera fase solo se establecen las entidades con sus atributos y cómo podría relacionarse mediante el verbo.



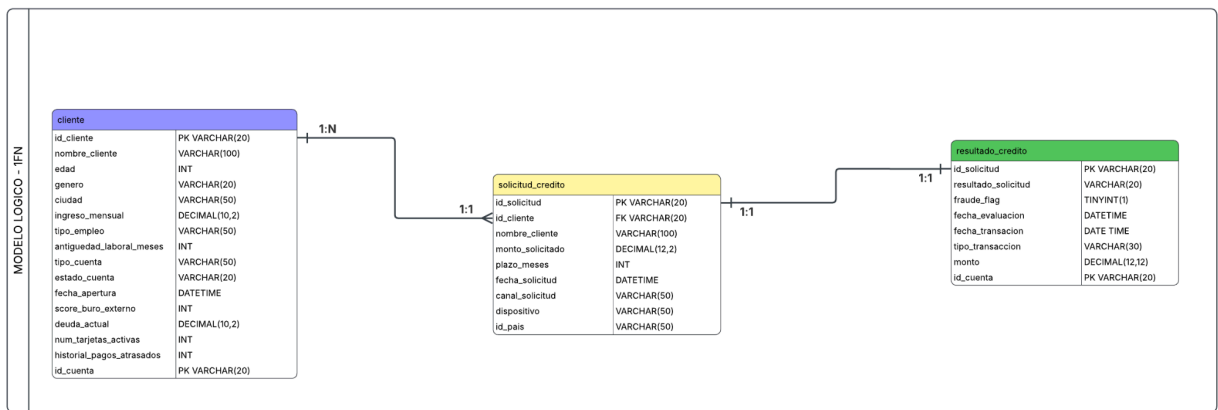
La relación se da; un cliente solicita un crédito, el mismo se evalúa para obtener un resultado sea APROBADO o RECHAZADO.

Modelo lógico

Iniciamos agregando la cardinalidad en las relaciones entre nuestras tablas, en la primera relación entre cliente y solicitud_credito, la relación quedaría de 1 a muchos, ya que un cliente puede hacer varias solicitudes de crédito, pero una solicitud de crédito es hecha por un solo cliente. En la segunda relación, está queda de uno a uno, ya que el resultado de cada solicitud sólo puede estar de una en una. Además, iniciamos a normalizar nuestra base de datos.

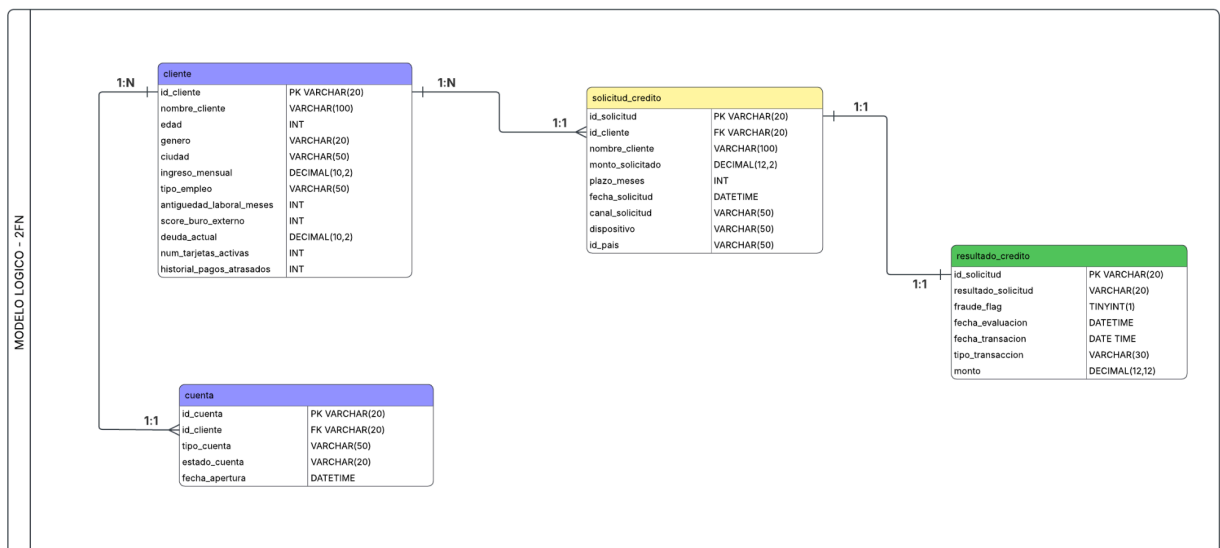
Aplicando 1FN

En esta fase de nuestro ML, agregamos la clave primaria que identifica cada registro en cada tabla. Además, establecemos los tipos de datos y sus longitudes a cada atributo, con eso evitamos que cada uno contenga repeticiones en sus datos. Con esto aplicado, va quedando nuestro ML de esta manera:



Aplicando 2FN

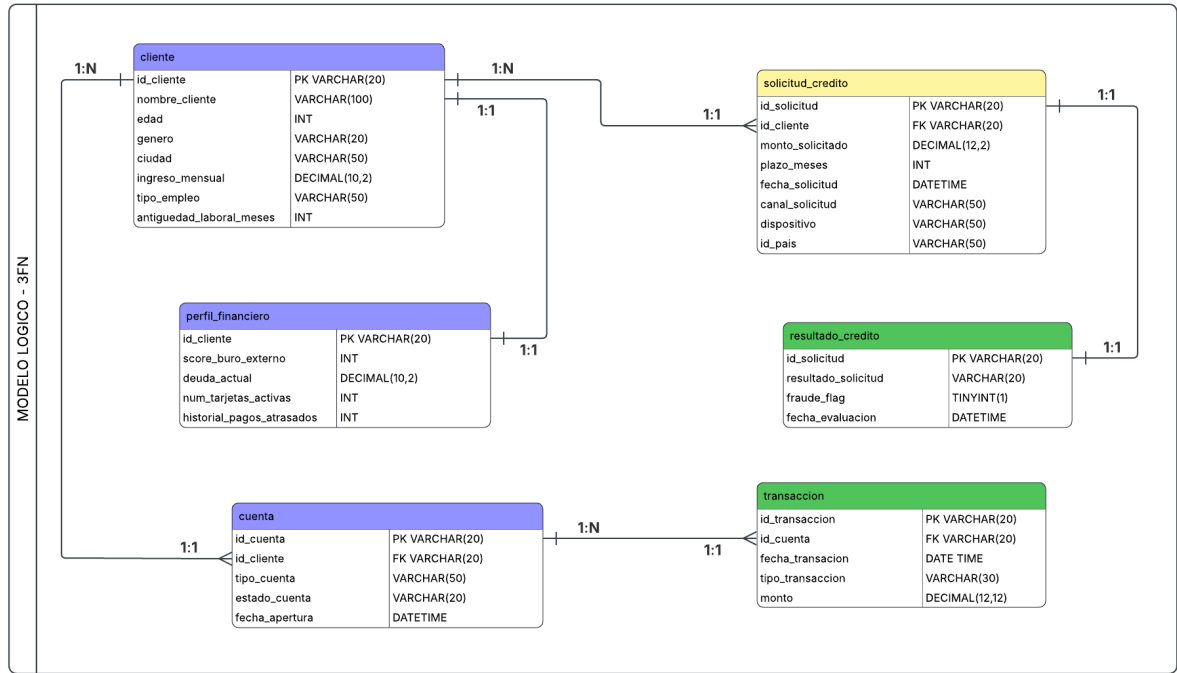
En este paso, observamos que existe una dependencia parcial en la tabla clientes, ya que el campo `id_cuenta` es un campo que puede ser clave primaria, además que, un cliente puede tener varias cuentas que puede usar para solicitar el crédito. Por tanto, se separó estos campos relacionados a cuentas bancarias en una tabla aparte, cuya relación se da de uno a muchos, ya que un cliente puede tener varias cuentas bancarias y una sola cuenta sólo pertenece a un único cliente. Por último, se elimina el campo `id_cuenta` de la tabla `resultado_solicitud` ya que al estar esos datos en otra tabla y relacionados de uno a uno, sería información redundante. Quedando de la siguiente manera.



Aplicando 3FN

En esta fase observamos que hay columnas que a pesar que conceptualmente pueden pertenecer a la tabla, no dependen de esta ni de otro campo. Por tanto, se optó por separar, al igual que la tabla cuentas, se creó la tabla `perfil_financiero`, con esto eliminamos las dependencia transitiva. Para ello, separamos los datos correspondientes y creamos la relación de uno a uno, ya que cada cliente tiene un solo perfil financiero y no depende de las cuentas que este tenga.

Otro cambio se dio en la tabla resultado_solicitud, en este hay campos que son para realizar el depósito del crédito solamente cuando es aprobado, esto se puede separar en otra tabla, ya que no están relacionadas explícitamente con los demás campos, más bien con la tabla de cuentas, así se elimina duplicar datos de otras tablas. Quedando de la siguiente manera.



Modelo físico

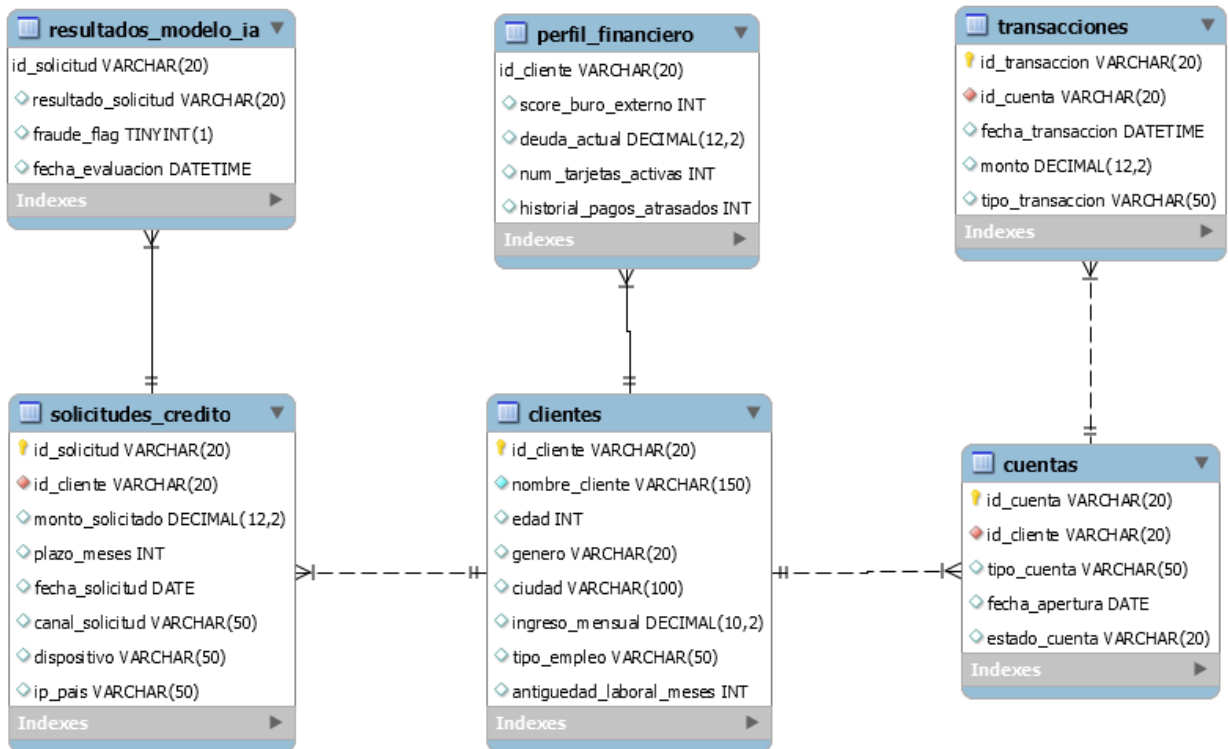
En este último paso, mediante el ML final creamos nuestro script de SQL para crear nuestra base de datos. Para esta práctica se usó el gestor de base de datos de MySQL.


```

1 -- 1. Entidad: Clientes (Datos demográficos estáticos)
2 CREATE TABLE Clientes (
3     id_cliente VARCHAR(20) PRIMARY KEY,
4     nombre_cliente VARCHAR(150) NOT NULL,
5     edad INT,
6     genero VARCHAR(20),
7     ciudad VARCHAR(100),
8     ingreso_mensual DECIMAL(10, 2),
9     tipo_empleo VARCHAR(50),
10    antigüedad_laboral_meses INT
11 );
12
13 -- 2. Entidad: Perfil_Financiero (Separado por 3FN)
14 CREATE TABLE Perfil_Financiero (
15     id_cliente VARCHAR(20) PRIMARY KEY,
16     score_buro_externo INT,
17     deuda_actual DECIMAL(12, 2),
18     num_tarjetas_activas INT,
19     historial_pagos_atrasados INT,
20     CONSTRAINT fk_perfil_cliente FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)
21 );
22
23 -- 3. Entidad: Cuentas
24 CREATE TABLE Cuentas (
25     id_cuenta VARCHAR(20) PRIMARY KEY,
26     id_cliente VARCHAR(20) NOT NULL,
27     tipo_cuenta VARCHAR(50),
28     fecha_apertura DATE,
29     estado_cuenta VARCHAR(20),
30     CONSTRAINT fk_cuenta_cliente FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)
31 );
32
33 -- 4. Entidad: Transacciones
34 CREATE TABLE Transacciones (
35     id_transaccion VARCHAR(20) PRIMARY KEY,
36     id_cuenta VARCHAR(20) NOT NULL,
37     fecha_transaccion DATETIME,
38     monto DECIMAL(12, 2),
39     tipo_transaccion VARCHAR(50),
40     CONSTRAINT fk_transaccion_cuenta FOREIGN KEY (id_cuenta) REFERENCES Cuentas(id_cuenta)
41 );
42
43 -- 5. Entidad: Solicitudes_Credito (Núcleo del dataset, sin los datos de IA)
44 CREATE TABLE Solicitudes_Credito (
45     id_solicitud VARCHAR(20) PRIMARY KEY,
46     id_cliente VARCHAR(20) NOT NULL,
47     monto_solicitado DECIMAL(12, 2),
48     plazo_meses INT,
49     fecha_solicitud DATE,
50     canal_solicitud VARCHAR(50),
51     dispositivo VARCHAR(50),
52     ip_pais VARCHAR(50),
53     CONSTRAINT fk_solicitud_cliente FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)
54 );
55
56 -- 6. Entidad: Resultados_Modelo_IA (Separado para garantizar auditoría y trazabilidad)
57 CREATE TABLE Resultados_Modelo_IA (
58     id_solicitud VARCHAR(20) PRIMARY KEY,
59     resultado_solicitud VARCHAR(20), -- Aprobado / Rechazado / Pendiente
60     fraude_flag TINYINT(1), -- 0 o 1
61     fecha_evaluacion DATETIME,
62     CONSTRAINT fk_resultado_solicitud FOREIGN KEY (id_solicitud) REFERENCES
63 Solicitudes_Credito(id_solicitud)
64

```

Después de ejecutar el script de SQL obtenemos el diagrama de entidad relación final de nuestro gestor de MySQL mediante el proceso de ingeniería inversa, siendo el siguiente:



3. Calidad y limpieza de datos

Limpiar un dataset ficticio de solicitudes de crédito (nulos, duplicados, outliers, tipos de datos inconsistentes), documentando cada decisión técnica. Definir reglas de calidad de datos que estas fuentes deberán cumplir de forma continua.

Para limpiar el dataset solicitudes_credito_neocredit.csv se va a utilizar las siguientes librerías:

```

1 !pip install pandera ydata-profiling missingno -q
2 import pandas as pd
3 import numpy as np
4 import missingno as msno
5 from sklearn.impute import SimpleImputer
6 from sklearn.preprocessing import (
7     MinMaxScaler, RobustScaler, MaxAbsScaler,
8     StandardScaler, Normalizer,
9 )
10 import pandera.pandas as pa
11 from pandera import Column, Check, DataFrameSchema
12 import matplotlib.pyplot as plt
13 import seaborn as sns
  
```

Perfilado y exploración inicial

Subimos el dataset a la nube de google colab e importamos. Observamos que la dimensión de nuestro dataset es de 669 filas y 21 columnas. Además, en un vistazo inicial observamos desde ya que hay columnas con datos faltantes; ya que el total de registros es de 669, pero hay columnas que tienen menos de este valor. Siendo `id_solicitud` el único campo que tiene todas las columnas.

```
1 df = pd.read_csv("/content/solicitudes_credito_neocredit.csv")
2 df.info()
```

Ahora mediante la estadística descriptiva de las columnas numéricas, observamos que existen valores negativos, estos observados en 3 de las 21 variables; lo cual también debemos considerar para nuestra limpieza de datos. Podemos observar que los datos faltantes ya están presentes desde nuestros primeros 10 registros.

```
1 df.describe()
2 df.head(10)
```

Detección de columnas irrelevantes

Antes de pasar al proceso del tratamiento con datos nulos, debemos observar si nuestro dataset tiene columnas irrelevantes. Para ello haremos conteo de cuántos subniveles (registros diferentes en cada columna). Se debe eliminar columnas que tengan menos de 2 subniveles, ya que al tener solo un mismo valor en todo no aporta nada al dataset para procesos de análisis. Para este dataset, observamos que todas las columnas tienen más de 1 subnivel, por tanto todas son relevantes tomando en cuenta este aspecto.

```
1 df_columnas = df.columns.tolist()
2 for col in df_columnas:
3     print(f'Columna {col}: {df[col].nunique()} subniveles')
```

```
Columna id_solicitud: 656 subniveles
Columna id_cliente: 529 subniveles
Columna nombre_cliente: 560 subniveles
Columna edad: 63 subniveles
Columna genero: 7 subniveles
Columna ciudad: 12 subniveles
Columna ingreso_mensual: 610 subniveles
Columna tipo_empleo: 8 subniveles
Columna antiguedad_laboral_meses: 223 subniveles
Columna monto_solicitado: 650 subniveles
Columna plazo_meses: 5 subniveles
Columna score_buro_externo: 376 subniveles
Columna deuda_actual: 650 subniveles
Columna num_tarjetas_activas: 7 subniveles
Columna historial_pagos_atrasados: 5 subniveles
Columna fecha_solicitud: 505 subniveles
Columna canal_solicitud: 7 subniveles
Columna dispositivo: 6 subniveles
Columna ip_pais: 7 subniveles
Columna resultado_solicitud: 5 subniveles
Columna fraude_flag: 2 subniveles
```

Se puede tomar la primera columna como irrelevante, ya que solo representa un código de transacción generado, aunque si observamos detalladamente, el total de registros distintos es 669, pero la columna `id_solicitud` tiene 656 subniveles distintos, es decir, hay solicitudes procesadas nuevamente. Sin embargo, concluimos que no estarían sumando algo importante para un posterior análisis, por tanto esta se eliminará, quedándonos con 20 columnas.

Hay otras columnas que a consideración nuestra podría tomarse como irrelevantes: nombre, género, ciudad y fecha_solicitud; pero al no ser expertos en temas de créditos y cómo esto podría sumar o no para un análisis posterior a la limpieza de datos, se optó por dejar todas las columnas.



```
1 df_i2 = df.iloc[:, 1:len(df.columns)]
2 df_i2.shape
```

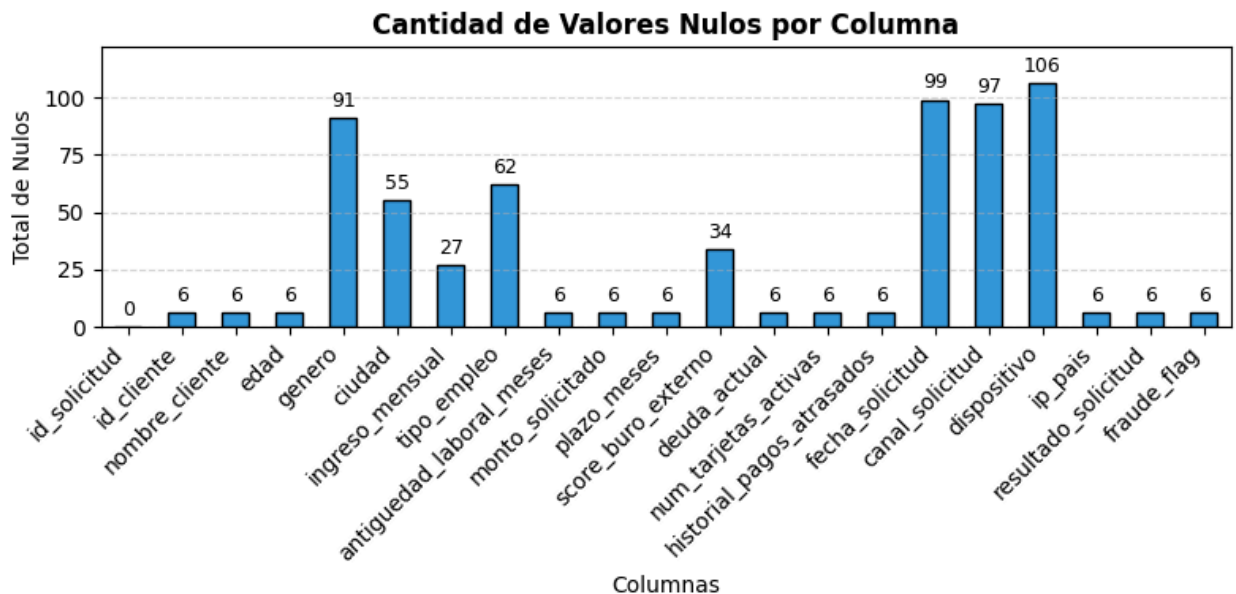
Tratamiento de datos faltantes

Antes de pasar a nuestro filtrado de datos faltantes, observamos mediante un conteo cuántos valores Nulos hay presentes por cada campo del dataset. Para esto usaremos la librería `matplotlib` para representar estos valores de manera gráfica y permite tener un mejor detalle de la cantidad de nulos.

```

1 import matplotlib.pyplot as plt
2
3 nulos = df.isnull().sum() #Obtener nulos
4 plt.figure(figsize=(8, 4)) #Crear figura
5 ax = nulos.plot(kind="bar", color="#3498db", edgecolor="black") #Guardar el gráfico en la variable 'ax'
6 ax.bar_label(ax.containers[0], padding=3, fontsize=9) #Agregar el conteo en cada barra
7 plt.title("Cantidad de Valores Nulos por Columna", fontsize=12, fontweight="bold") #Personalización de ejes y título
8 plt.xlabel("Columnas")
9 plt.ylabel("Total de Nulos")
10 plt.xticks(rotation=45, ha="right")
11 plt.grid(axis="y", linestyle="--", alpha=0.5)
12 plt.ylim(0, nulos.max() * 1.15) #Dar un poco más de espacio arriba para que el número no toque el borde del gráfico
13 plt.tight_layout()
14 plt.show()

```



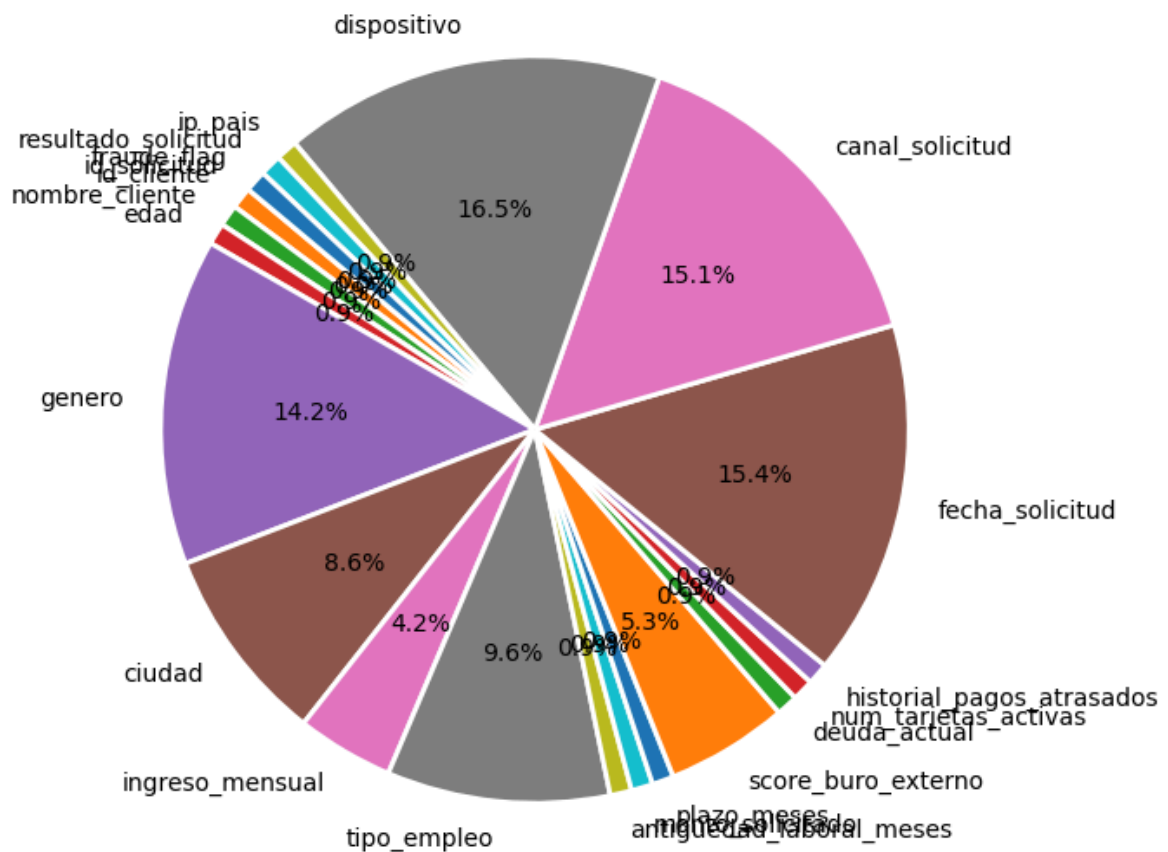
Al igual que el gráfico anterior obtuvimos el total de registros nulos por columnas, también observaremos en el gráfico a continuación cuánto representa en porcentaje estos nulos en nuestro dataset. Con esto, podremos tener un análisis inicial del impacto de los datos faltantes en nuestro dataset.

```

1 plt.figure(figsize=(7, 7))
2 plt.pie(#Generar el gráfico de pastel con porcentajes
3     nulos,
4     labels=nulos.index, #Nombres de las columnas
5     autopct="%1.1f%%", #Muestra solo el porcentaje
6     startangle=140,
7     wedgeprops={"edgecolor": "white", "linewidth": 2})
8 plt.title("Proporción de Valores Nulos por Columna", fontsize=12, fontweight="bold")
9 plt.show()

```

Proporción de Valores Nulos por Columna



Hemos analizado las cantidades y porcentajes de valores nulos presentes por columna en el dataset pero, también debemos observar el porcentaje representado por fila, siendo este más grande aún, siendo exactamente del 58.15% del total del dataset, más de la mitad al menos tiene un valor nulo en cada fila. Esto nos da a entender definitivamente que no se debe realizar la eliminación de los valores faltantes. Para este caso debemos aplicar técnicas de imputación de datos.

```
1 print(f"El porcentaje de nulos por fila es: {(((df.isna().any(axis=1).sum()) / len(df)) * 100).round(2)}%")
```

Imputación de datos numéricos

Como se explicó en la sección anterior; no se realizó la eliminación de datos faltantes debido a que representaban un alto porcentaje tanto para filas como sus columnas. Se realizará el proceso de imputación de datos mediante la mediana en cada columna. Pero, en primer lugar debemos realizar la evaluación de que los tipos de datos sean correctos, acorde a lo esperado por la columna. Observamos algunas como ingreso_mensual que debería ser numérica pero está como tipo objeto. Estas conversiones se realizan para los datos numéricos.

Los casos como la edad, plazo_meses, antigüedad_laboral_meses y otros más ya están como numéricos float, pero sabemos que las edades o meses no se representan como decimales, por ello se transformará a entero. Adicionalmente, se ha detectado que hay valores de tipo texto en las columnas de tipo numérico, esto se da debido que tiene valores con signos como el \$, se hará la eliminación y transformación de esos valores.

```
1 df_i2_i4 = df_i2.copy()
2
3 columnas = ['edad', 'antigüedad_laboral_meses', 'plazo_meses', 'num_tarjetas_activas', 'historial_pagos_atrasados', 'fraude_flag'] #a enteros
4 for col in columnas:
5     df_i2_i4[col] = df_i2_i4[col].astype(str).str.replace(r'[\$]', '', regex=True)
6     df_i2_i4[col] = pd.to_numeric(df_i2_i4[col], errors='coerce').astype('Int64')
7
8 columnas = ['ingreso_mensual', 'monto_solicitado', 'score_buro_externo', 'deuda_actual'] #a decimales
9 for col in columnas:
10    df_i2_i4[col] = df_i2_i4[col].astype(str).str.replace(r'[\$]', '', regex=True)
11    df_i2_i4[col] = pd.to_numeric(df_i2_i4[col], errors='coerce')
```

Después de realizar la conversión de los tipos de datos, mediante la función info() comprobamos los cambios efectuados correctamente.

```
RangeIndex: 669 entries, 0 to 668
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id_cliente                           663 non-null    object
1   nombre_cliente                       663 non-null    object
2   edad                                 663 non-null    Int64
3   genero                               578 non-null    object
4   ciudad                              614 non-null    object
5   ingreso_mensual                      586 non-null    float64
6   tipo_empleo                          607 non-null    object
7   antigüedad_laboral_meses             663 non-null    Int64
8   monto_solicitado                     663 non-null    float64
9   plazo_meses                          663 non-null    Int64
10  score_buro_externo                   635 non-null    float64
11  deuda_actual                         663 non-null    float64
12  num_tarjetas_activas                 663 non-null    Int64
13  historial_pagos_atrasados             663 non-null    Int64
14  fecha_solicitud                      570 non-null    object
15  canal_solicitud                      572 non-null    object
16  dispositivo                          563 non-null    object
17  ip_pais                              663 non-null    object
18  resultado_solicitud                  663 non-null    object
19  fraude_flag                          663 non-null    Int64
dtypes: Int64(6), float64(4), object(10)
memory usage: 108.6+ KB
```

Ahora procederemos con la imputación de datos por cada columna, para realizar este proceso se ha optado por rellenar esos "huecos" con la mediana de cada columna; ¿Por Qué este método y no usar el método de relleno hacia atrás (bfill)? En la práctica y acorde a investigaciones, este método se emplea más para series o datos ordenados, como nuestro conjunto de datos no tiene secuencias en sus columnas o información que sea serializada no es conveniente aplicar este

método. Por ello, haremos uso de la estrategia de imputación con la mediana utilizando el método SimpleImputer.

Seleccionamos las filas que son numéricas para realizar el proceso mencionado anteriormente.

```
1 columnas = ['edad', 'ingreso_mensual', 'antiguedad_laboral_meses', 'monto_solicitado', 'plazo_meses', #numericas
2             'score_buro_externo', 'deuda_actual', 'num_tarjetas_activas', 'historial_pagos_atrasados', 'fraude_flag']
3 imputer = SimpleImputer(strategy="median")
4 for col in columnas:
5     df_i2_i4[col] = imputer.fit_transform(df_i2_i4[[col]])
```

Errores tipográficos

Para verificar errores tipográficos en variables categóricas, debemos observar los subniveles de cada columna para validar si se pueden hacer correcciones. Como ejemplo, tenemos la columna género; ésta solamente registra 2 géneros: Masculino y Femenino, sin embargo, en el dataset hay abreviaturas y en mayúsculas o minúsculas, dando esto más subniveles que representan los mismos.

Se realizará una reducción de subniveles que sean iguales o que estén tipados erróneamente. Primero realizaremos este proceso en las variables categóricas. No se toma en cuenta: id_cliente, nombre_cliente y fecha_solicitud debido que son valores que no son categorizados.

```
1 df_i2_i4_i5 = df_i2_i4.copy()
2 print(f"La columna genero tiene: {df_i2_i4['genero'].unique()}")
3 print(f"La columna ciudad tiene: {df_i2_i4['ciudad'].unique()}")
4 print(f"La columna tipo_empleo tiene: {df_i2_i4['tipo_empleo'].unique()}")
5 print(f"La columna canal_solicitud tiene: {df_i2_i4['canal_solicitud'].unique()}")
6 print(f"La columna dispositivo tiene: {df_i2_i4['dispositivo'].unique()}")
7 print(f"La columna ip_pais tiene: {df_i2_i4['ip_pais'].unique()}")
8 print(f"La columna resultado_solicitud tiene: {df_i2_i4['resultado_solicitud'].unique()}")
```

```
La columna genero tiene: [nan 'Femenino' 'Masculino' 'm' 'F' 'f' 'Otro' 'M']
La columna ciudad tiene: [nan 'Cuenca' 'Ibarra' 'quito' 'Manta' 'Loja' 'Ambato' 'Quito' 'Guayaquil'
'GUAYAQUIL' 'Machala' 'Sto. Domingo' 'Cuenca']
La columna tipo_empleo tiene: ['Empleado' 'Jubilado' 'independiente' nan 'EMPLEADO' 'Independiente'
'estudiante' 'empleado' 'Desempleado']
La columna canal_solicitud tiene: ['Call Center' nan 'web' 'APP' 'app movil' 'Sucursal' 'App' 'Web']
La columna dispositivo tiene: ['IOS' 'Android' nan 'Desconocido' 'iOS' 'android' 'Desktop']
La columna ip_pais tiene: ['Ecuador' 'Rusia' 'Colombia' 'Peru' nan 'Ecuador (VPN)' 'Nigeria'
'Vietnam']
La columna resultado_solicitud tiene: ['Aprobado' 'Pendiente' 'Rechazado' 'aprobado' 'RECHAZADO' nan]
```

Como se observa en el resultado anterior, en cada columna existen categorías que se repiten de diferente forma, vamos a hacer la unificación de subniveles. Para este caso práctico, al haber categorías que son iguales pero se diferencian en mayúsculas o minúsculas, se consideró dejar todos los valores en minúsculas. Con esto podemos reducir el número de categorías en las columnas.


```

1 columnas = ['genero', 'ciudad', 'tipo_empleo', 'canal_solicitud', 'dispositivo', 'ip_pais', 'resultado_solicitud']
2 for col in columnas:
3     df_i2_i4_i5[col] = df_i2_i4_i5[col].str.lower()
4
5 print(f"La columna genero tiene: {df_i2_i4_i5['genero'].unique()}")
6 print(f"La columna ciudad tiene: {df_i2_i4_i5['ciudad'].unique()}")
7 print(f"La columna tipo_empleo tiene: {df_i2_i4_i5['tipo_empleo'].unique()}")
8 print(f"La columna canal_solicitud tiene: {df_i2_i4_i5['canal_solicitud'].unique()}")
9 print(f"La columna dispositivo tiene: {df_i2_i4_i5['dispositivo'].unique()}")
10 print(f"La columna ip_pais tiene: {df_i2_i4_i5['ip_pais'].unique()}")
11 print(f"La columna resultado_solicitud tiene: {df_i2_i4_i5['resultado_solicitud'].unique()}")

```

```

La columna genero tiene: [nan 'femenino' 'masculino' 'otro']
La columna ciudad tiene: [nan 'cuenca' 'ibarra' 'quito' 'manta' 'loja' 'ambato' 'guayaquil'
'machala' 'sto. domingo']
La columna tipo_empleo tiene: ['empleado' 'jubilado' 'independiente' nan 'estudiante' 'desempleado']
La columna canal_solicitud tiene: ['call center' nan 'web' 'app' 'sucursal']
La columna dispositivo tiene: ['ios' 'android' nan 'desconocido' 'desktop']
La columna ip_pais tiene: ['ecuador' 'rusia' 'colombia' 'peru' nan 'ecuador (vpn)' 'nigeria'
'vietnam']
La columna resultado_solicitud tiene: ['aprobado' 'pendiente' 'rechazado' nan]

```

Ahora observamos que en la columna ciudad, hay una categoría que tiene espacios al final, vamos a aplicar a todas las columnas la eliminación de espacios al inicio y al final; además con esto bajamos los subniveles de las columnas.

```

1 for col in columnas:
2     df_i2_i4_i5[col] = df_i2_i4_i5[col].str.strip()

```

Como último paso, modificamos los subniveles por columna con el fin de reducir y unificarlos. En este caso solamente aplicamos en la columna género y canal_solicitud ya que es la que más presentan subniveles repetidos.

```

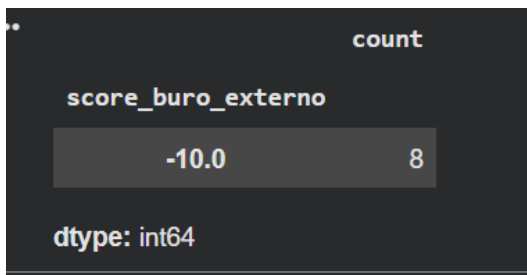
1 df_i2_i4_i5.loc[df_i2_i4_i5['genero'] == 'm', 'genero'] = 'masculino'
2 df_i2_i4_i5.loc[df_i2_i4_i5['genero'] == 'f', 'genero'] = 'femenino'
3 df_i2_i4_i5.loc[df_i2_i4_i5['canal_solicitud'] == 'app movil', 'canal_solicitud'] = 'app'

```

Como vimos en el ítem del perfilado de datos, observamos que existen datos en las columnas numéricas como negativos, esto puede considerarse como un error tipográfico, ¿Porqué lo consideramos así? en las columnas en la que se vio estos datos son: edad, ingreso_mensual y antigüedad_laboral_mes, se analiza y realmente se puede notar que son variables que no pueden ser negativas ya que no se puede tener una edad negativa, nadie tampoco gana en menos, el tiempo en meses no es negativo. El único campo que consideramos dejará en negativo es el score_buro_externo debido que no sabemos como es la lógica de la entidad para clasificar el buro de solicitante, por tanto se dejan los valores negativos presentes (En entornos normales no

deberían existir). Pero analizando un poco, realizamos el conteo de los diferentes valores negativos en esta columna y se observó que solo hay un valor presente en varios registros, por tanto es un valor asignado por la misma entidad para clasificar a sus solicitantes.

```
1 df_i2_i4_i5[df_i2_i4_i5['score_buro_externo'] < 0]['score_buro_externo'].value_counts()
```



Realizamos la transformación de valores negativos en las 3 columnas mencionadas.

```
1 df_i2_i4_i5['edad'] = df_i2_i4_i5['edad'].abs()  
2 df_i2_i4_i5['ingreso_mensual'] = df_i2_i4_i5['ingreso_mensual'].abs()  
3 df_i2_i4_i5['antiguedad_laboral_meses'] = df_i2_i4_i5['antiguedad_laboral_meses'].abs()  
4 df_i2_i4_i5.describe()
```

Imputación de datos categóricos

Para este proceso se toma en consideración el porcentaje de valores nulos presentes en cada columna categórica, ya que no se aplica el mismo proceso que en las variables numéricas. Para ello hemos considerado que; Si el porcentaje de Nan es menor al 5% se usará la moda para llenar los espacios vacíos. Si el porcentaje de Nan es mayor al 5% se llenará con la categoría "desconocido".

Tomando eso en cuenta, y basándonos en el gráfico de pastel del porcentaje de nulos presentes, las columnas categóricas que se imputarán con la moda son:

- id_cliente
- nombre_cliente
- id_pais
- resultado_solicitud

Y las columnas que se imputará con la categoría "desconocido" son:

- genero
- ciudad

- tipo_empleo
- fecha_solicitud
- canal_solicitud
- dispositivo

```
1 df_i2_i4_i5_i6 = df_i2_i4_i5.copy()
2 columnas = ['id_cliente', 'nombre_cliente', 'ip_pais', 'resultado_solicitud']
3 for col in columnas:
4     moda = df_i2_i4_i5_i6[col].mode()[0]
5     df_i2_i4_i5_i6[col] = df_i2_i4_i5_i6[col].fillna(modas)
6
7 columnas = ['genero', 'ciudad', 'tipo_empleo', 'fecha_solicitud', 'canal_solicitud',
8             'dispositivo']
9 for col in columnas:
10    df_i2_i4_i5_i6[col] = df_i2_i4_i5_i6[col].fillna('desconocido')
```

Al realizar el último proceso de imputación observamos con la función info() que los datos en su totalidad en cada columna están completos.

```
1 df_i2_i4_i5_i6.info()
```

```
RangeIndex: 669 entries, 0 to 668
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id_cliente                           669 non-null    object
1   nombre_cliente                       669 non-null    object
2   edad                                 669 non-null    float64
3   genero                               669 non-null    object
4   ciudad                               669 non-null    object
5   ingreso_mensual                      669 non-null    float64
6   tipo_empleo                          669 non-null    object
7   antiguedad_laboral_meses             669 non-null    float64
8   monto_solicitado                     669 non-null    float64
9   plazo_meses                          669 non-null    float64
10  score_buro_externo                   669 non-null    float64
11  deuda_actual                         669 non-null    float64
12  num_tarjetas_activas                  669 non-null    float64
13  historial_pagos_atrasados             669 non-null    float64
14  fecha_solicitud                       669 non-null    object
15  canal_solicitud                       669 non-null    object
16  dispositivo                           669 non-null    object
17  ip_pais                              669 non-null    object
18  resultado_solicitud                   669 non-null    object
19  fraude_flag                           669 non-null    float64
dtypes: float64(10), object(10)
memory usage: 104.7+ KB
```

Escalado y normalización

Para el escalado de los datos se optó por el método de escalado robusto (RobustScaler) ya que nuestros datos tienen valores atípicos que si usamos el escalado estándar (MinMaxScaler) puede verse afectada la proporción de los valores.

```
1 df_i2_i4_i5_i6_i7 = df_i2_i4_i5_i6.copy()
2 columnas = ['edad', 'ingreso_mensual', 'antiguedad_laboral_meses', 'monto_solicitado', 'plazo_meses', #numericas
3             'score_buro_externo', 'deuda_actual', 'num_tarjetas_activas', 'historial_pagos_atrasados']
4 robust_scaler = RobustScaler()
5 for col in columnas:
6     df_i2_i4_i5_i6_i7[col + "robust_scaler"] = robust_scaler.fit_transform(df_i2_i4_i5_i6_i7[[col]])
```

Para la normalización, se optó por utilizar los 2 métodos enseñados en clases mediante los talleres, para que sea de práctica y al final quede un dataset resultante con todas las variantes para aplicar al análisis de datos posterior al proceso de calidad y limpieza de datos.

```
1 pt = PowerTransformer(method="yeo-johnson")
2 l2_normalizer = Normalizer(norm="l2")
3 for col in columnas:
4     df_i2_i4_i5_i6_i7[col + "yeo_norm"] = pt.fit_transform(df_i2_i4_i5_i6_i7[[col]])
5     df_i2_i4_i5_i6_i7[col + "l2_norm"] = l2_normalizer.fit_transform(df_i2_i4_i5_i6_i7[[col]])
```

Definición de reglas para la calidad de los datos

Como punto final, validamos manualmente con reglas de calidad de los datos, comprobando si el resultado final cumple con los esperados en cada columna; como el tipo de datos esperado, si hay nulos, valores únicos y algunas reglas de negocio. Validamos que el resultado final cumple con los requisitos esperados. Estas son definidas a continuación:

Columna	Tipo	Nulo	Único	longitud min/max	valores entre	Formato
id_cliente	String	No	Si	-	-	CLI-
nombre_cliente	String	No	-	>=2 <=100	-	-
edad	Float	Si	-	-	[0, 90]	-
genero	String	Si	-	-	-	masculino femenino otro desconocido
ciudad	String	Si	-	>=2	-	-

ingreso_mensual	Float	Si	-	-	≥ 0	-
tipo_empleo	String	Si	-	-	-	empleado jubilado independiente estudiante desempleado desconocido
antiguedad_laboral_mes	Float	Si	-	-	[0,100 0]	-
monto_solicitado	Float	Si	-	-	-	-
plazo_meses	Float	Si	-	-	-	-
score_buro_externo	Float	Si	-	-	[-10,10 00]	-
deuda_actual	Float	Si	-	-	≥ 0	-
num_tarjetas_activas	Float	Si	-	-	[0,25]	-
historial_pagos_atrasados	Float	Si	-	-	≥ 0	-
fecha_solicitud	String	Si	-	-	-	YYYY-MM-DD
canal_solicitud	String	Si	-	-	-	call center web app sucursal desconocido
dispositivo	String	Si	-	-	-	ios android desktop desconocido
ip_pais	String	Si	-	≥ 2	-	-
resultado_solicitud	String	Si	-	-	-	aprobado pendiente rechazado
fraude_flag	Float	Si	-	-	[0,1]	-

Esto evaluamos con el código siguiente en python usando la librería de pandas.

```

1 import pandera.pandas as pa
2 schema = DataFrameSchema(
3     columns={
4         "id_cliente": Column(
5             str,
6             Check.str_matches(r"^\d{10}$"),
7             nullable=False,
8             unique=True,
9         ),
10        "nombre_cliente": Column(
11            str,
12            Check.str_length(
13                min_value=2, max_value=100
14            ),
15            nullable=True,
16        ),
17        "edad": Column(
18            float,
19            Check.between(0, 210),
20            nullable=True,
21        ),
22        "genero": Column(
23            str,
24            # Lista cerrada de opciones válidas
25            Check.isin(["masculino", "femenino", "otro", "desconocido"]),
26            nullable=True,
27        ),
28        "ciudad": Column(
29            str,
30            Check.str_length(min_value=2),
31            nullable=True,
32        ),
33        # 3. Información Laboral y Financiera
34        "ingreso_mensual": Column(
35            float,
36            Check.ge(0), # Mayor o igual a cero
37            nullable=True,
38        ),
39        "tipo_empleo": Column(
40            str,
41            Check.isin(
42                ['empleado', 'jubilado', 'independiente', 'estudiante', 'desempleado', 'desconocido']
43            ),
44            nullable=True,
45        ),
46        "antigüedad_laboral_meses": Column(
47            float,
48            Check.between(0, 10000), # Máximo 50 años de antigüedad laboral
49            nullable=True,
50        ),
51        # 4. Comportamiento Crediticio (Buró)
52        "score_buro_externo": Column(
53            float,
54            # Típico rango de score crediticio (ej. 300 a 850 ó 1 a 1000)
55            Check.between(-10, 850),
56            nullable=True,
57        ),
58        "deuda_actual": Column(
59            float,
60            Check.ge(0),
61            nullable=True,
62        ),
63        "num_tarjetas_activas": Column(
64            float,
65            Check.between(0, 25), # Límite lógico de tarjetas por persona
66            nullable=True,
67        ),
68        "historial_pagos_atrasados": Column(
69            float,
70            Check.ge(0), # Conteo de moras
71            nullable=True,
72        ),
73    },

```

```

1      # 5. Datos de la Solicitud y Canal
2      "fecha_solicitud": Column(
3          # Convertir la columna a datetime en Pandas o validarla como formato ISO
4          str,
5          Check.str_matches(
6              r"^\d{4}-\d{2}-\d{2}"
7          ), # Garantiza formato YYYY-MM-DD
8          nullable=True,
9      ),
10     "canal_solicitud": Column(
11         str,
12         Check.isin(['call center', 'web', 'app', 'sucursal', 'desconocido']),
13         nullable=True,
14     ),
15     "dispositivo": Column(
16         str,
17         Check.isin(['ios', 'android', 'desconocido', 'desktop']),
18         nullable=True,
19     ),
20     "ip_pais": Column(
21         str,
22         Check.str_length(min_value=2),
23         nullable=True,
24     ),
25     # 6. Resultados y Fraude
26     "resultado_solicitud": Column(
27         str,
28         Check.isin(['aprobado', 'pendiente', 'rechazado']),
29         nullable=True,
30     ),
31     "fraude_flag": Column(
32         float,
33         # Al ser un indicador (flag), solo debe tomar valores 0 (No) o 1 (Sí)
34         Check.isin([0.0, 1.0]),
35         nullable=True,
36     ),
37 },
38 # Regla de coherencia cruzada
39 checks=[
40     # Un desempleado no debería tener 20 años de antigüedad laboral activa
41     Check(
42         lambda df: ~(
43             (df["tipo_empleo"] == "Desempleado")
44             & (df["antigüedad_laboral_meses"] > 0)
45         ),
46         name="check_coherencia_desempleado_antigüedad",
47     )
48 ],
49 )
50 try:
51     schema.validate(df_i2_i4_i5_i6_i7, lazy=True)
52     print("El dataset cumple con las reglas de calidad definidas.")
53 except pa.errors.SchemaErrors as err:
54     print(err.failure_cases)

```

Exportación del dataset limpio

Una vez aplicada las técnicas de calidad y limpieza de datos, podemos exportar el data frame en un nuevo archivo .csv para que pueda ser utilizado en nuevos procesos de análisis de datos.

```

1 ruta = '/content/neo_credit_clean.csv'
2 df_i2_i4_i5_i6_i7.to_csv(ruta, index=False)

```

4. Metadatos y trazabilidad

Para asegurar la transparencia, auditabilidad y confianza en el modelo de Inteligencia Artificial desarrollado para NeoCredit, es fundamental estructurar un esquema de metadatos robusto. Este esquema actúa como una "etiqueta nutricional" o Model Card, documentando tanto los datos de entrenamiento extraídos de nuestra arquitectura híbrida, como las características operativas del modelo final.

Antes de realizar el esquema de metadatos se realizó una injección de datos para garantizar que se esté realizando correctamente la trazabilidad de los datos.

77	INSERT INTO Clientes (id_cliente, nombre_cliente, edad, genero, ciudad, ingreso_mensual, tipo_empleo, antiguedad_laboral_meses) VALUES	
78	('CLI001', 'Ana Garcia Lopez', 34, 'Femenino', 'Ciudad de Mexico', 2500.00, 'Asalariado', 48),	
79	('CLI002', 'Juan Perez Gomez', 28, 'Masculino', 'Bogota', 1800.00, 'Independiente', 24),	
80	('CLI003', 'Maria Rodriguez Silva', 45, 'Femenino', 'Lima', 3200.50, 'Asalariado', 120);	
81		
82	INSERT INTO Perfil_Financiero (id_cliente, score_buro_externo, deuda_actual, num_tarjetas_activas, historial_pagos_atrasados) VALUES	
83	('CLI001', 720, 1500.00, 2, 0),	
84	('CLI002', 580, 3000.00, 4, 3),	
85	('CLI003', 810, 500.00, 1, 0);	
86		
87	INSERT INTO Cuentas (id_cuenta, id_cliente, tipo_cuenta, fecha_apertura, estado_cuenta) VALUES	
88	('CTA001', 'CLI001', 'Corriente', '2023-01-15', 'Activa'),	
89	('CTA002', 'CLI002', 'Ahorros', '2024-05-20', 'Activa'),	
90	('CTA003', 'CLI003', 'Corriente', '2020-11-10', 'Activa');	
91		
92	INSERT INTO Transacciones (id_transaccion, id_cuenta, fecha_transaccion, monto, tipo_transaccion) VALUES	
93	('TRX001', 'CTA001', '2026-07-20 10:15:00', -150.00, 'Retiro'),	
94	('TRX002', 'CTA001', '2026-07-22 14:30:00', 500.00, 'Deposito'),	
95	('TRX003', 'CTA002', '2026-07-23 09:00:00', -800.00, 'Pago Servicio');	
96		
97	INSERT INTO Solicitudes_Credito (id_solicitud, id_cliente, monto_solicitado, plazo_meses, fecha_solicitud, canal_solicitud, dispositivo, ip_pais) VALUES	
98	('SOL001', 'CLI001', 5000.00, 24, '2026-07-21', 'App Movil', 'iOS', 'Mexico'),	
99	('SOL002', 'CLI002', 12000.00, 48, '2026-07-22', 'Web', 'Windows', 'Colombia'),	
100	('SOL003', 'CLI003', 2000.00, 12, '2026-07-23', 'App Movil', 'Android', 'Peru');	
101		
102	INSERT INTO Resultados_Modelo_IA (id_solicitud, resultado_solicitud, fraude_flag, fecha_evaluacion) VALUES	
103	('SOL001', 'Aprobado', 0, '2026-07-21 10:05:00'),	
104	('SOL002', 'Rechazado', 1, '2026-07-22 11:30:00');	
105	('SOL003', 'Pendiente', 0, '2026-07-23 09:15:00');	

Message	Summary	Profile	Status
Query			
INSERT INTO Clientes (id_cliente, nombre_cliente, edad, genero, ciudad, ingreso_mensual, tipo_empleo, antiguedad_laboral_meses) VALUES ('CLI001', 'Ana Garcia Lopez', 34, 'Femenino', 'Ciudad de Mexico', 2500.00, 'Asalariado', 48), ('CLI002', 'Juan Perez Gomez', 28, 'Masculino', 'Bogota', 1800.00, 'Independiente', 24), ('CLI003', 'Maria Rodriguez Silva', 45, 'Femenino', 'Lima', 3200.50, 'Asalariado', 120)		Message	Query Time
		Affected rows: 3	0.033s
INSERT INTO Perfil_Financiero (id_cliente, score_buro_externo, deuda_actual, num_tarjetas_activas, historial_pagos_atrasados) VALUES ('CLI001', 720, 1500.00, 2, 0), ('CLI002', 580, 3000.00, 4, 3), ('CLI003', 810, 500.00, 1, 0)		Affected rows: 3	0.003s
INSERT INTO Cuentas (id_cuenta, id_cliente, tipo_cuenta, fecha_apertura, estado_cuenta) VALUES ('CTA001', 'CLI001', 'Corriente', '2023-01-15', 'Activa'), ('CTA002', 'CLI002', 'Ahorros', '2024-05-20', 'Activa'), ('CTA003', 'CLI003', 'Corriente', '2020-11-10', 'Activa')		Affected rows: 3	0.003s
INSERT INTO Transacciones (id_transaccion, id_cuenta, fecha_transaccion, monto, tipo_transaccion) VALUES ('TRX001', 'CTA001', '2026-07-20 10:15:00', -150.00, 'Retiro'), ('TRX002', 'CTA001', '2026-07-22 14:30:00', 500.00, 'Deposito'), ('TRX003', 'CTA002', '2026-07-23 09:00:00', -800.00, 'Pago Servicio')		Affected rows: 3	0.003s
INSERT INTO Solicitudes_Credito (id_solicitud, id_cliente, monto_solicitado, plazo_meses, fecha_solicitud, canal_solicitud, dispositivo, ip_pais) VALUES ('SOL001', 'CLI001', 5000.00, 24, '2026-07-21', 'App Movil', 'iOS', 'Mexico'), ('SOL002', 'CLI002', 12000.00, 48, '2026-07-22', 'Web', 'Windows', 'Colombia'), ('SOL003', 'CLI003', 2000.00, 12, '2026-07-23', 'App Movil', 'Android', 'Peru')		Affected rows: 3	0.003s
Elapsed Time: 0.077s			

Esquema de Metadatos (Model Card y Data Card)

Se ha diseñado un esquema que clasifica la información en tres dimensiones fundamentales: descriptiva, estructural y administrativa, abarcando el ecosistema de IA de NeoCredit.

1. Metadatos Descriptivos (Contexto y Propósito) Describen el "qué" y el "para qué" del conjunto de datos y del modelo, orientados al negocio.

Nombre del Modelo: NeoCredit_Risk_Scoring_XGBoost

Versión: v2.1.4

Fecha de Lanzamiento: 2026-07-24

Propósito : Predecir la probabilidad de incumplimiento de pago (default) y el riesgo de fraude en solicitudes de microcréditos para clientes en Latinoamérica, automatizando la decisión de aprobación, rechazo o revisión manual.

Descripción del Dataset de Entrenamiento: Datos combinados. Estructurados e históricos (demografía, perfil financiero, transacciones) extraídos del núcleo transaccional en MySQL. Datos no estructurados y semiestructurados (comportamiento en app, interacciones de soporte) extraídos de Firestore.

Limitaciones (Limitations): El modelo está calibrado específicamente para microcréditos de consumo. No es aplicable a créditos hipotecarios o empresariales. Requiere reentrenamiento trimestral para mitigar el *concept drift* por factores macroeconómicos.

2. Metadatos Estructurales (Arquitectura y Formato) Definen la composición técnica de los datos en origen y las características internas del algoritmo.

Arquitectura del Modelo: para clasificación binaria y evaluación de riesgos múltiples.

Formato de Entrada (Features): Vector de variables continuas y categóricas.

Variables SQL: ingreso_mensual, deuda_actual, historial_pagos_atrasados (provenientes de las tablas Clientes y Perfil_Financiero).

Variables NoSQL: tiempo_sesion_segundos (de comportamiento_app), categoria_sentimiento (de soporte_cliente).

3. Extracción de Metadatos Estructurales (SQL): Para documentar la procedencia de los datos estructurados, se interroga el diccionario nativo de MySQL (INFORMATION_SCHEMA) sobre la base de datos neocredit:

```
-- Consulta 1: Inventario general de tablas de la base de datos neocredit
-- (Te muestra qué tablas existen, cuántas filas tienen y el motor).
SELECT TABLE_NAME, TABLE_ROWS, ENGINE, CREATE_TIME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'neocredit';
```

Message	Summary	Result 1	Profile	Status
TABLE_NAME		TABLE_ROWS	ENGINE	CREATE_TIME
▶ clientes		3	InnoDB	2026-07-23 23:35:43
cuentas		3	InnoDB	2026-07-23 23:35:43
perfil_financiero		3	InnoDB	2026-07-23 23:35:43
resultados_modelo_ia		3	InnoDB	2026-07-23 23:35:43
solicitudes_credito		3	InnoDB	2026-07-23 23:35:43
transacciones		3	InnoDB	2026-07-23 23:35:43

```
-- Consulta 2: Diccionario de datos (Estructura) de la tabla "Clientes"
-- (Útil para mostrar el tipo de dato y restricciones de la tabla principal).
SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE, COLUMN_KEY
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'neocredit' AND TABLE_NAME = 'Clientes';
```

131 -- Consulta 4: Localizar columnas sensibles (PII) para la

Message	Summary	Result 1	Profile	Status
	COLUMN_NAME	DATA_TYPE	IS_NULLABLE	COLUMN_KEY
►	id_cliente	varchar	NO	PRI
	nombre_cliente	varchar	NO	
	edad	int	YES	
	genero	varchar	YES	
	ciudad	varchar	YES	
	ingreso_mensual	decimal	YES	
	tipo_empleo	varchar	YES	
	antiguedad_laboral_r	int	YES	

-- Consulta 3: Diccionario de datos de la tabla "Solicitudes_Credito"
 -- (Muestra los campos clave del negocio antes de pasar por la IA).

```
SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE, COLUMN_KEY
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'neocredit' AND TABLE_NAME = 'Solicitudes_Credito';
```

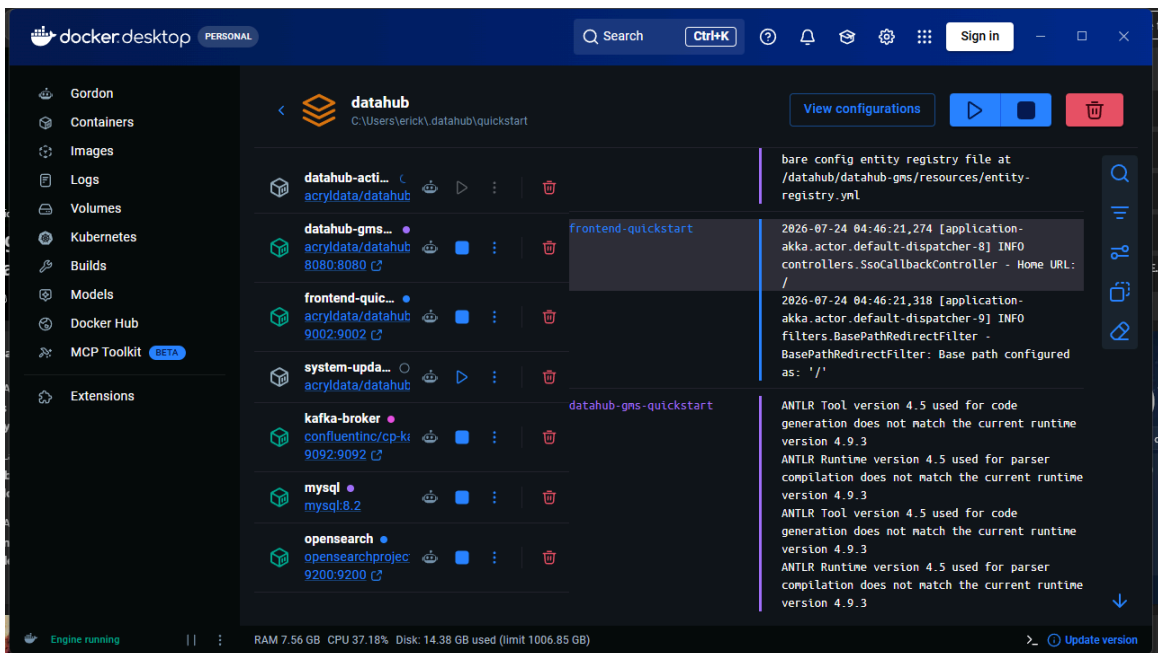
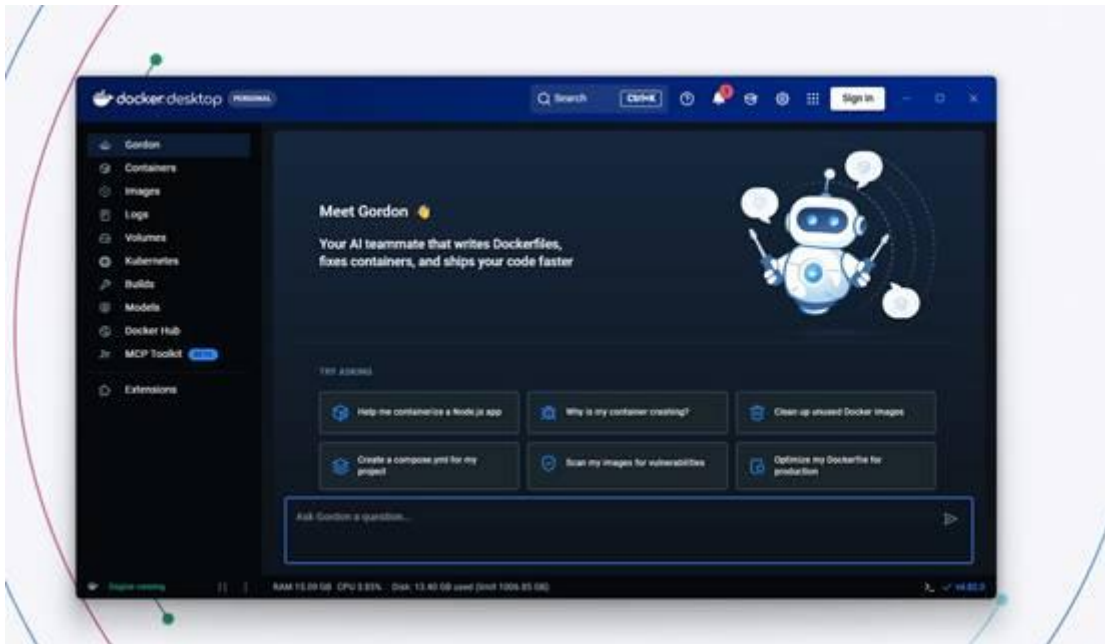
Message	Summary	Result 1	Profile	Status
	COLUMN_NAME	DATA_TYPE	IS_NULLABLE	COLUMN_KEY
►	id_solicitud	varchar	NO	PRI
	id_cliente	varchar	NO	MUL
	monto_solicitado	decimal	YES	
	plazo_meses	int	YES	
	fecha_solicitud	date	YES	
	canal_solicitud	varchar	YES	
	dispositivo	varchar	YES	
	ip_pais	varchar	YES	

-- Consulta 4: Localizar columnas sensibles (PII) para la gobernanza y etiquetado
 -- (Demuestra cómo buscarías datos que requieren protección antes de usar la IA).

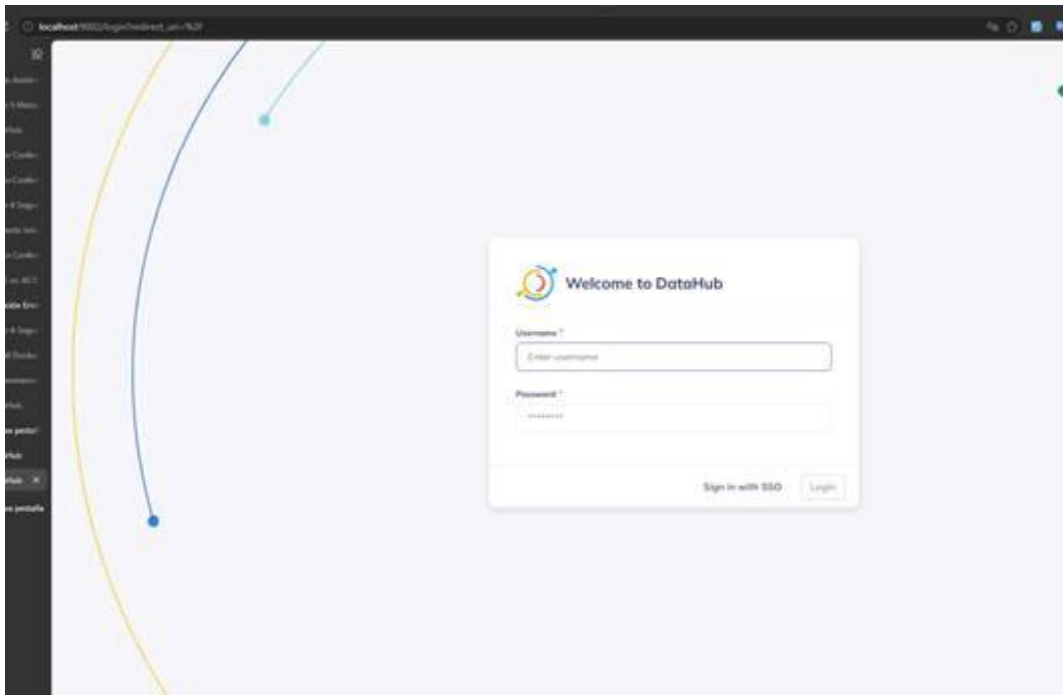
```
SELECT TABLE_NAME, COLUMN_NAME, DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'neocredit'
AND (COLUMN_NAME LIKE '%correo%'
OR COLUMN_NAME LIKE '%telefono%'
OR COLUMN_NAME LIKE '%nombre%'
OR COLUMN_NAME LIKE '%direccion%'
OR COLUMN_NAME LIKE '%ciudad%');
```

Message	Summary	Result 1	Profile	Status
	TABLE_NAME	COLUMN_NAME	DATA_TYPE	
►	clientes	nombre_cliente	varchar	
	clientes	ciudad	varchar	

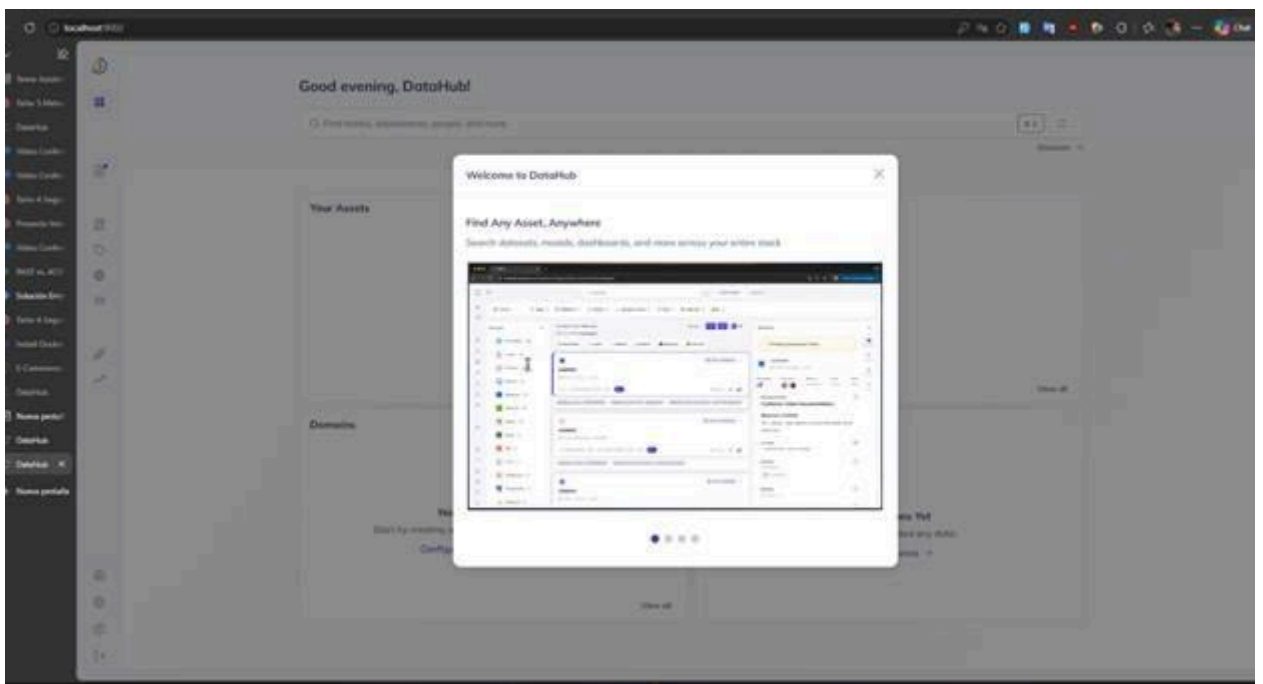
Parte 2: Levantar DataHub con Docker



Se muestra la interfaz de Docker Desktop con la herramienta "Gordon" (un asistente de IA para Docker). Esta sección indica que se han inicializado y levantado los contenedores necesarios en el entorno local para ejecutar la plataforma de gobierno de datos **DataHub**.



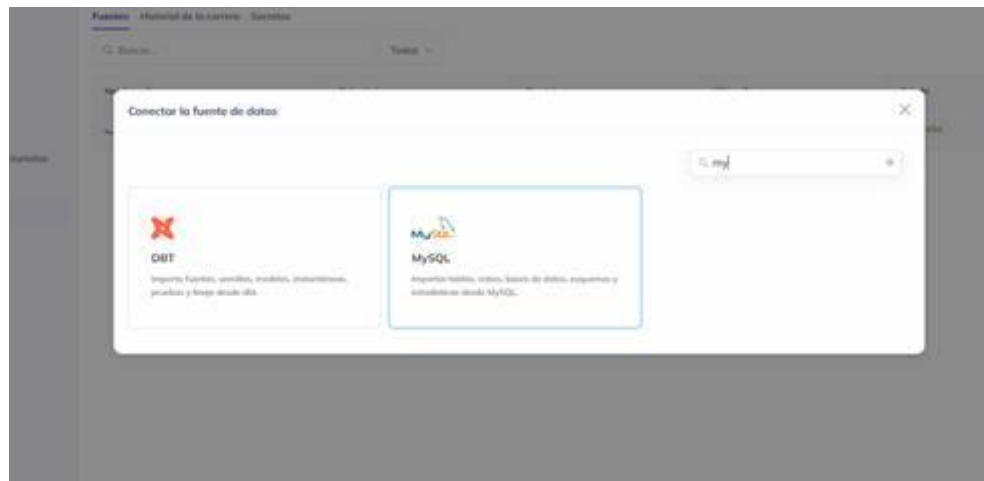
Se abrió el navegador web apuntando a localhost:9002 para acceder a la interfaz gráfica de DataHub, mostrando la pantalla oficial de inicio de sesión (Welcome to DataHub).



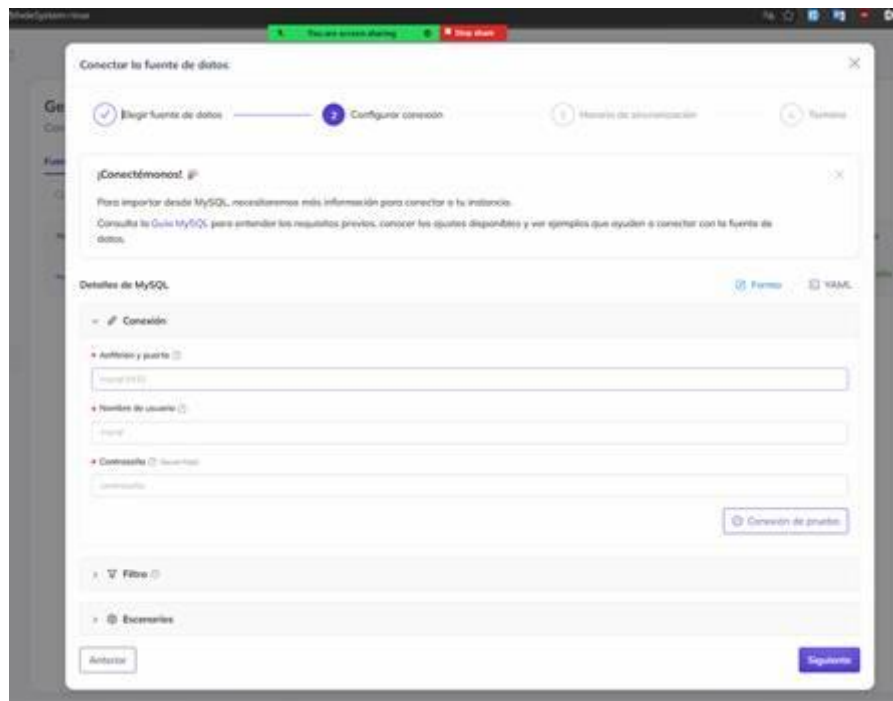
Muestra el primer ingreso exitoso a la plataforma, donde aparece una ventana emergente de bienvenida que guía al usuario sobre cómo buscar cualquier activo de datos dentro del catálogo.

Parte 3: Recolectar los metadatos de neocredit

Se selecciona la fuente de datos.



Se agregan las credenciales de datos.



5. Backup y recuperación

Estrategia de Respaldo para Datos Críticos

Para garantizar la integridad y disponibilidad de las tablas SQL del sistema, se ha diseñado una estrategia de respaldo utilizando Duplicati, considerando la exportación previa de la base de datos o el respaldo de sus archivos de datos (.sql).

1. RespalDOS Completos (Full Backup):

Frecuencia: Semanal (Todos los domingos a las 02:00 AM).

Propósito: Crear una línea base completa (baseline) de toda la base de datos SQL.

2. RespalDOS Incrementales / Modificaciones de Bloque:

Frecuencia: Diaria (Lunes a Sábado a las 23:00 PM).

Mecanismo con Duplicati: Duplicati detecta automáticamente los bloques de datos modificados desde la última copia de seguridad. Esto optimiza el uso de almacenamiento y ancho de banda mediante duplicación y compresión.

3. Estrategia Sintética Diferencial:

Gracias al motor de duplicación de Duplicati, cada punto de restauración actúa como una versión independiente sin necesidad de acumular cadenas complejas de incrementales, reduciendo el riesgo de corrupción en la cadena de restauración.

Objetivos de Tiempo y Punto de Recuperación (RTO y RPO)

1. RPO (Recovery Point Objective - Objetivo de Punto de Recuperación):

Meta: 24 horas (Máxima pérdida de datos tolerable).

Justificación: Al ejecutarse un respaldo automático cada 24 horas, en caso de fallo crítico, la pérdida máxima de transacciones o registros SQL será de un día de trabajo.

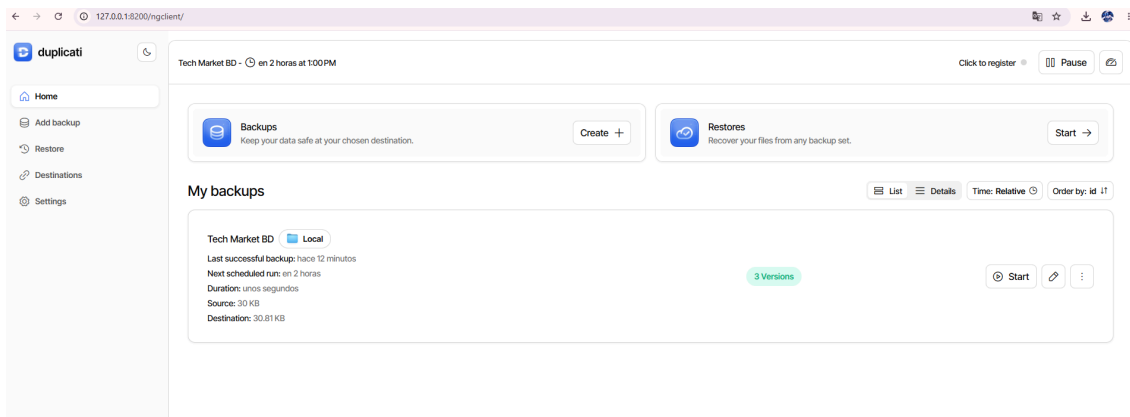
2. RTO (Recovery Time Objective - Objetivo de Tiempo de Recuperación):

Meta: 1 a 2 horas (Tiempo máximo para restaurar el servicio).

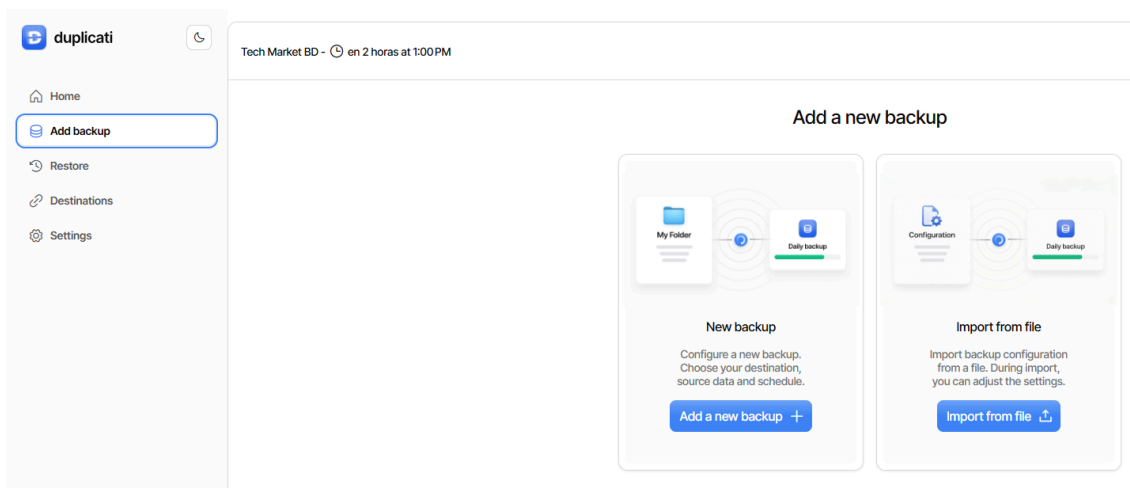
Justificación: Considerando el tamaño estimado de la base de datos y la velocidad de descompresión/descarga local desde el almacenamiento de respaldo, el tiempo de reconstrucción y ejecución del restore en la base de datos toma menos de 120 minutos.

Paso a Paso en Duplicati

1. Se abre la interfaz web de Duplicati: Se abre el navegador de preferencia y se ingresa a la URL local predeterminada: <http://localhost:8200> para ingresar al panel de control principal.



2. **Se selecciona la opción de añadir una nueva copia:** En el menú lateral izquierdo, se hace clic en **Añadir copia de seguridad**, se selecciona **Configurar una nueva copia de seguridad** y se presiona el botón **Siguiente**.



3. **Se configuran el nombre del respaldo y la contraseña de cifrado:** Se asigna el nombre Backup_NEOCREDIT, se deja seleccionado el tipo de cifrado **Cifrado AES-256 de Duplicati** y se establece una frase de contraseña segura para proteger los datos. Se hace clic en **Siguiente**.

General backup settings

Backup name *

Backup_NEOCREDIT

Backup description

Copia de seguridad de los datos de la BD NeoCredit

Encryption

Cifrado AES-256, incorporado

Create a password *

Show

Generate password

Tip: Be sure to save your password. If you lose it, you will lose access to your backups.

Exit

Continue

4. **Se define la ubicación de destino del respaldo:** En la sección de **Destino**, se selecciona el **Tipo de almacenamiento** (*Carpeta local*) y se define la ruta del directorio donde se guardarán los paquetes del respaldo. Se presiona **Siguiente**.

File system

Store backups on your local file system.

Change

☐ Save as connection string

Make sure to test the configuration

Test now

Browse path

Manually type path

File path

Hogwarts Legacy

MAESTRIA

NEOCREDIT

BACKUP_NEOCREDIT

Plantillas personalizadas de Office

PROFORMAS AMBULANCIAS

RSTUDIO

☐ Show hidden folders

Create folder

Advanced options

Add advanced option

5. **Se seleccionan los datos de origen (Archivo .sql):** En el árbol de directorios de **Datos de origen**, se navega hasta ubicar y marcar la casilla de la carpeta donde MySQL Workbench guarda el archivo .sql. Se hace clic en **Siguiente**.

Tech Market BD - en 2 horas at 1:00 PM

Click to register Pause

General Destination Source Data Schedule Options

Source Data

Folder path Show hidden files

Computer

- My Documents
 - Arduino
 - Hogwarts Legacy
 - MAESTRIA
 - NEOCREDIT
 - BACKUP_NEOCREDIT
 - neocredit (1).sql (2.3 KB)**
 - Plantillas personalizadas de Office
 - PROFORMAS AMBULANCIAS
 - RSTUDIO
 - SQL SCRIPTS
 - The Witcher 3
 - Zoom
 - .biblio_gemini_model.txt (19 Bytes)
 - .biblio_graph_settings.txt (17 Bytes)
 - .RData (6 KB)
 - .Rhistory (6.69 KB)
 - 2026.docx (728.72 KB)

Paths

Manually add a path.

Add local disk Add remote path

%MY_DOCUMENTS%\NEOCREDIT\neocredit (1).sql

Add a direct path

Filters

Customize your filters.

+ Add filter

Exclude

Choose if there's any files you want to exclude.

Files larger than

Go back Continue

6. **Se programa el horario de ejecución (Cumplimiento de RPO):** En la sección **Horario**, se marca la opción **Ejecutar automáticamente**, se establece la hora (ejemplo: 23:00) y se selecciona la frecuencia **Diariamente** para garantizar el cumplimiento del RPO de 24 horas.

Schedule

Custom

☒ Automatically run backups

If a date was missed, the job will run as soon as possible.

Next time

23:00 Jul 21, 2026

Run again every

1 Days

Allowed days

☒ Monday ☒ Tuesday

☒ Wednesday ☒ Thursday

☒ Friday ☒ Saturday

☒ Sunday

Go back Continue

7. **Se ajustan las opciones avanzadas y la retención:** En **Retención de copia de seguridad** se selecciona *Conservar todas las copias de seguridad*). Se hace clic en **Guardar**.

Options

Backup retention

Keep all backups

Nothing will be deleted. The backup size will grow with each change.

Advanced options

Add advanced option +



Add a setting above to get started.

← Go back

→ Submit

8. **Se realiza la ejecución manual inicial:** En el menú principal, se selecciona el trabajo recién creado (Backup_NEOCREDIT) y se hace clic en **Ejecutar ahora** para iniciar la primera copia de seguridad completa y validar su correcto funcionamiento.

Backup_NEOCREDIT

Local

Last successful backup: hace unos segundos

Next scheduled run: en 12 horas

Duration: unos segundos

Source: 2.3 KB

Destination: 3.83 KB

1 Version

Start

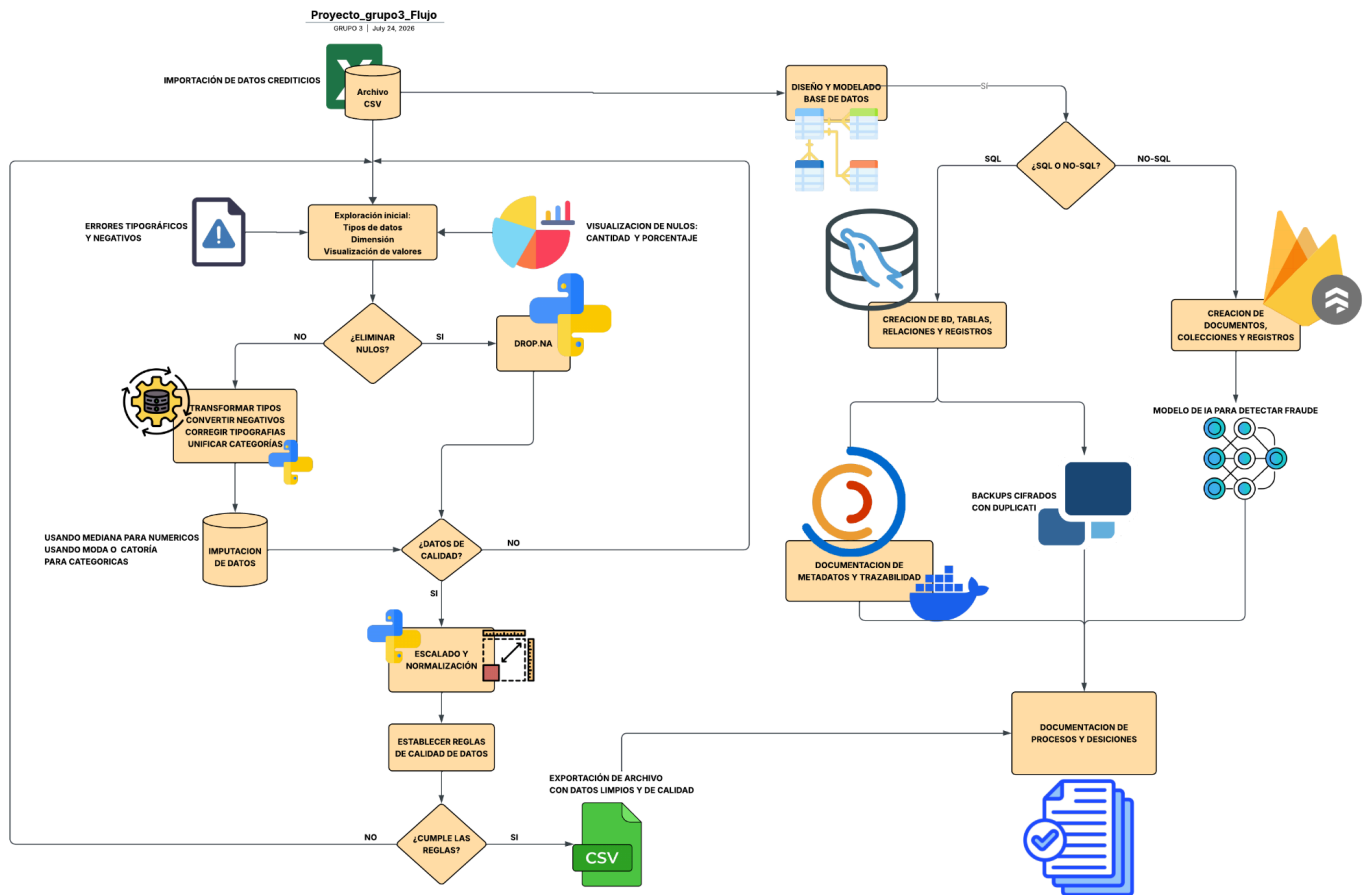
9. **Verificación de los archivos generados en el directorio de destino:** Se navega hasta la carpeta Documentos/NEOCREDIT/BACKUP_NEOCREDIT para confirmar la creación exitosa de los volúmenes cifrados (.dlist.zip.aes, .dblock.zip.aes y .dindex.zip.aes).

Inicio de copia > Documentos > NEOCREDIT > BACKUP_NEOCREDIT				
Ordenar Ver ...				
Nombre	Fecha de modificación	Tipo	Tamaño	
duplicati-20260722T155502Z.dlist.zip.aes	22/07/2026 10:55	Archivo AES	2 KB	
duplicati-b5e56be375ab34a5796b341e3009de331.dblock.zip.aes	22/07/2026 10:55	Archivo AES	2 KB	
duplicati-i013c434cfc874ead89f4123cfa3f9f7e.dindex.zip.aes	22/07/2026 10:55	Archivo AES	1 KB	

Como se pudo palpar en este paso, los backups son la columna vertebral de la continuidad de negocio en bases de datos: garantizan la integridad de la información y minimizan el impacto crítico ante fallos, ataques o pérdidas de datos. Asegurar una alta disponibilidad a través de copias de seguridad continuas previene interrupciones operativas y pérdidas financieras devastadoras. Como se vió en este taller, el uso de herramientas como Duplicati optimizan este proceso al ofrecer automatización, cifrado y almacenamiento eficiente para mantener los datos siempre protegidos y recuperables.

6. Documentación y diagrama de flujo de datos

Se realizó un diagrama de flujo el cual representa de manera resumida los pasos y decisiones tomadas en cada proceso, el mismo se elaboró en la herramienta Lucidchart.



En este diagrama se integra las herramientas, recursos y decisiones tomadas en cada sección anterior, desde la limpieza de los datos, del porqué se dio de esa manera la limpieza y validación de la calidad de los datos. La elección de los tipos de gestores de datos, SQL y NoSQL; ambas opciones se pueden integrar en un solo sistema híbrido el cual se acoplan bien para procesos de backups y documentación de metadatos como en casos de creación de modelos de IA (con las BD NoSQL).

Es importante señalar que el conocimiento en el manejo de todas las herramientas vistas en este proyecto son de vital importancia para la gobernabilidad de los datos, desde el diseño y modelado hasta las copias de seguridad, se integran en un solo conjunto de herramientas que desconoce de tier list, todas están a nivel de importancia similar.

En la actualidad, obtener documentación veraz de los metadatos ayuda mucho a los profesionales a realizar los procesos de análisis, calidad y limpieza de datos de manera ágil, al no tener que depender de terceros o personas que ya no estén en el entorno. Lo que implica entregar informes estratégicos en menos tiempo que ayudan a la toma de decisiones, mismas que impactan positivamente en cualquier negocio o institución.

7. Repositorio Github

https://github.com/cburbano-usgp/gestion_datos_proyecto_grupo3.git